

Generation

Define los Criterios para la Tarea de la IA y Escribe los Prompts

Paso 1: Definir los Criterios Específicos para la Tarea (Marco CLEAR)

Utiliza el marco CLEAR para definir los criterios específicos que tu herramienta de IA debe seguir al analizar los archivos de registro.

1. Relevancia del Contexto

- ¿Qué alcance debe tener la IA? ¿Hay tipos específicos de registros o palabras clave que se deben priorizar?
- Ejemplo: "Enfocarse en errores y advertencias relacionadas con la actividad del usuario; excluye los registros informativos a menos que estén marcados como críticos."

**Enfocarse en leer y entender los archivos locales, de aplicaciones y redes.
Tiene que buscar principalmente los errores, fallos y advertencias que afecten el sistema, dejando mensajes informativos en caso de que sirva para entender un fallo.**

2. Coincidencia de Tono y Estilo del Lenguaje

- ¿Qué tono debe usar la IA? (por ejemplo, técnico, amigable)

Usar un tono claro, directo, y educativo, explicando las cosas de forma ordenada para que yo pueda entender y aprender mientras reviso el código o informes.

3. Manejo de Errores y Escalamiento

- ¿Cómo debe responder la IA si hay incertidumbre o falta de datos?

Si hay falta de datos o información o los registros son confusos, la IA no debe inventar respuestas. Sino que debe ser honesta y que diga las cosas como son: "No encuentro suficiente información en los logs para darte una respuesta segura"

4. Precisión y Fiabilidad de las Respuestas

- ¿Cómo asegurarás que las respuestas de la IA sean confiables y basadas en datos?

Todo lo que la IA responda tiene que estar basada 100% en los datos reales, que se encuentran en los archivos de registro, sin suponer nada.

5. Eficiencia de las Respuestas

- ¿Qué tipos de preguntas debe ser capaz de manejar la IA? ¿Cómo debe adaptar sus respuestas según la complejidad de la pregunta?

Debe ser capaz de responder tanto preguntas rápidas (como por ejemplo, buscar un error puntual) como peticiones más grandes (como resumir varios errores), adaptando el tamaño de su respuesta para que no me sobrecargue o me abrume.

Paso 2: Escribir los Prompts (Marco TRACI)

Usar el marco TRACI para redactar 1-3 prompts que dirijan la herramienta de IA a realizar cada tarea.

- Tarea 1:

Prompt 1

Prompt 1: Resumir y tabular errores principales

Lee los registros y dame una lista con los 5 errores que más se repiten. Actúa como un compañero programador que me ayuda a entender mis errores. Para un programador aprendiz. Pon la información en una tabla sencilla que muestre el tipo de error y cuántas veces pasó. Y saber rápido qué está fallando para poder corregirlo.

Prompt 2

Prompt 2: Para comparar actualizaciones o detectar patrones

Compara estos registros nuevos con los anteriores para ver si mi cambio arregló el problema o si trajo errores nuevos. Actúa como un tutor de programación paciente. Para un estudiante que quiere aprender a probar su código. Haz una lista corta con puntos que diga qué se arregló y qué sigue fallando. Aprender a verificar si mis correcciones de software funcionan de verdad.

Prompt 3

Prompt 3: Para analizar incidentes de seguridad y actividad de red

Revisa los registros buscando accesos extraños, intentos de inicio de sesión fallidos o problemas en la red. Actúa como un profesor que explica las cosas paso a paso. Para mí, para entender lo básico de seguridad en sistemas. Escribe un reporte corto con la hora del problema y una explicación muy fácil de entender. Aprender a detectar actividades sospechosas en un servidor.

Repítelo para cada tarea.

Plantilla de Documentación

Instrucciones:

1. Introducción

- Propósito de la Guía

Documentar el proceso de como crear y configurar mi propio asistente de IA para que me ayude a leer y entender archivo de registro (log files), atrapar errores rápido y aprender a depurar código mientras avanzo en mis estudios de programación.

- Alcance

Explica todo el proceso que seguí: desde definir qué quería que hiciera la IA y establecer reglas (usando los marcos CLEAR y TRACI), hasta crear el perfil en la plataforma, probarlo con fragmentos de código con errores reales y dejarlo listo para usar.

2. Descripción de la Herramienta

- Descripción

Es un agente de IA personalizado que funciona como un compañero o tutor de programación. Su trabajo principal es revisar textos de logs de servidores y aplicaciones para explicarme de forma muy clara qué está fallando.

- Características Principales

- Lee y analiza archivos de registros web y de servidores sin enredarse.
- Cuenta y organiza en tablas los errores que más se repiten.
- Explica los problemas paso a paso con un lenguaje fácil de digerir para alguien que está aprendiendo.
- Usa un tono amigable, directo y claro.

3. Proceso de Desarrollo

Guía Paso a Paso

1. Establecimiento de Objetivos

Pensar exactamente que quería que hiciera la IA (resumir errores, encontrar patrones en los logs y agrupar fallos comunes) y ponerle reglas claras sobre cómo debería responder y que hacer si no sabía algo.

2. Recopilación de Datos

Juntar ejemplos de código con errores típicos de sintaxis y lógica, así como pequeños fragmentos de registro de servidores para usarlos como pruebas de la IA.

3. Procesamiento previo de Datos

Organizar mis ideas usando el truco del marco TRACI (tarea, rol, audiencia, crear, intención) para armar los prompts exactos que la IA iba a entender sin dar vueltas.

4. Entrenamiento y Refinamiento de la Herramienta

Meter el perfil y las plantillas en la plataforma de IA, probar con los códigos malos de muestra y corregir las instrucciones las veces que fuera necesario hasta que la respuesta saliera limpia y ordenada en tablas.

5. Pruebas y Evaluación

Comprobar que la IA no se inventara cosas cuando los registros estuvieran incompletos y que realmente me ayudara a entender el problema para corregirlo en mi código.

4. Principales Desafíos y Soluciones

- **Desafío 1**
 - El problema: Al principio, la IA me daba respuestas larguísimas y aburridas que no me decían exactamente dónde estaba el error en el código.
 - La solución: Le cambié el prompt, le di un rol de "compañero programador" y le pedí directo que me armara tablas cortas con los errores.
 - **Desafío 2**
 - El problema: A veces, si el log estaba incompleto, la IA intentaba adivinar y me daba respuestas falsas.
 - La solución: Le puse una regla estricta en su configuración (el criterio de manejo de errores) para que mejor me diga "No hay suficiente información" en lugar de inventar.
 - **Consejos para el Éxito**
 - Sé súper claro y directo cuando le pidas algo a la IA.
 - Haz pruebas reales con código tuyo antes de dar por buena tu plantilla.
-

5. Casos de Uso Potenciales

- **Más Allá del Análisis de Registros**
 - Me sirve para hacer una depuración de código más rápido de scripts chicos en Python o JavaScript.
 - Me ayuda a entender por qué me sale un error raro en la terminal mientras hago mis tareas de programación.
 - **Oportunidades Futuras**
 - En el futuro, conectar la IA a una base de datos con todos los errores que ya he solucionado para que me sirva de acordeón personal.
-

6. Instrucciones para el Usuario

- **Cómo Usar la Herramienta**
 - Entra a la plataforma y activa el perfil que creamos.
 - Pega el texto del log o el pedazo de código que te está dando problemas.
 - Usa la plantilla ("Lucio, el analista") de *prompt* para pedirle que te liste y explique los errores.

- **Entradas y Salidas**

- Entrada: Un archivo de texto con registros o un código con errores.
- Salida: Una tabla limpia con los errores más repetidos y una explicación sencilla de cómo arreglarlos.

- **Consejos para el Usuario**

- Pega solo la parte del registro donde sospechas que está el fallo; si le metes demasiada información basura, la IA se confunde.
-

7. Escenarios de Ejemplo

- **Escenario 1:** Probar un script de JavaScript con variables mal escritas para ver cómo la IA me hace la listita en tabla de lo que falló.
- **Escenario 2:** Comparar los registros antes y después de modificar mi código para ver si mi corrección realmente solucionó el problema o si dejó vivos los errores.